

EMUQU — A Measurement-Grounded Assistant with a Multilingual Regulated-Claim Perimeter

Typed fact catalogue, runtime number verification, and one 17-language vocabulary enforced at input, output and build time

Technical White Paper • v1.2

v1.2 (2026-10-08): restructured method-first; adds design rationale; validation detail moved to the separate validation report.

October 2026

Chris Sharp

Describes Emuqu at source revision e028039 (2026-10-07); scoring version v3.1.oct2026.

Executive Summary

Emuqu is an iOS heart-rate-variability and training app with a conversational coach ("Flo") that runs on six language-model providers: five hosted (Anthropic, OpenAI, Google Gemini, xAI Grok, DeepSeek) and one on-device (Apple Intelligence). The coach has two jobs beyond being helpful. It must quote the user's own measurements correctly: "your TSB is -29" while the dashboard shows -15.5 is an error the user can check. And it must stay inside a wellness perimeter in every shipped language: no naming of a condition for the user, no rhythm assessment, no treatment or dosing, no categorical certainty about physiological state, while never leaving a user without help when they report an emergency, an injury or thoughts of self-harm.

Emuqu solves both with deterministic, on-device mechanisms rather than instructions to the model:

1. **A typed fact catalogue.** Every fact the model can read is one declarative entry binding a key grammar, a description, a value type, an availability check and the resolver closure, so a resolver cannot exist without being published. Results share one JSON envelope with a closed, nine-value absence taxonomy across all six providers.
2. **A compact tool layer.** 21 polymorphic read tools and 18 allow-listed action tools, de-duplicated and sorted for byte stability, with a capability index generated from the same specification list.
3. **Runtime number verification.** Claims about a metric's value *now* are checked against app state with per-metric tolerances, after spelled-out numbers are normalised. Only the claimed span is corrected, and the correction feeds the next turn's prompt.
4. **Structural conflict resolution.** When the user contradicts cached numbers, the stale dashboard block is removed from the prompt rather than argued with.

5. **One multilingual regulated-claim lexicon.** 29 concepts in 17 languages drive a pre-send input guard that answers locally with fixed, localized replies; a sentence-level, negation-neutral output guard applied before display and speech; and a build-time linter over Swift source and every locale of the string catalogues. A CI gate proves, with a witness string per pattern, that the runtime lexicon covers the build-time list.

Section 4 states what is new as fourteen specific rules, each with the public work it builds on and its earliest dated record.

1. The Problem

1.1 Numbers that must match the dashboard

Language models produce fluent text that can contradict their inputs, the failure the hallucination literature calls intrinsic or input-conflicting hallucination [1]. In a health-data assistant it appears as checkable errors: in logs this app recorded, a model quoted training stress balance (TSB) values from −9 to −29 across turns while the dashboard showed one stable value, and in voice mode a model stated 72 bpm while the live heart rate was 91. The causes differ:

- **Guessing instead of retrieving**, from priors or an earlier turn.
- **Disagreeing sources**, such as a readiness snapshot frozen at workout start versus the current load.
- **Stale context**: a cached prompt block contradicts what the user just said. Benchmarks of stale-premise rejection suggest prose instructions ("trust the user") are not reliable.
- **Spoken numbers**: in voice mode the model writes "negative thirty-one point seven", which a digit-matching check does not see.

Retrieval-augmented generation [2] and tool use [3][4] reduce guessing. RARR [5] and FActScore [6] verify claims post hoc with further model calls. Proof-Carrying Numbers verifies claim-bound numeric tokens against an authoritative source under a tolerance policy, failing closed [18], and US Patent 12,670,149 detects and corrects inaccurate numeric comparisons in LLM responses [19]. A streaming voice coach adds two requirements: the check must run in milliseconds on each sentence of free text before it is spoken, and it must read spelled-out numbers.

1.2 A regulated perimeter in seventeen languages

The app is a general-wellness product and not a medical device. US FDA general-wellness guidance [7] and App Store Review Guideline 1.4.1 [8] both turn on what the product claims. "You may have atrial fibrillation" is a claim the app cannot support; "see a doctor" without a crisis line fails a user in crisis. The perimeter must therefore:

- **not rest on the system prompt alone**, since prompt rules can be ignored, truncated or overridden by injection [9];
- **run on the device, between token arrival and speech**, where classifier guardrails such as Llama Guard [10] (Llama Guard 3: eight languages, categories including Specialized Advice and Self-Harm [22]) and rail frameworks such as NeMo Guardrails [11] (streaming output rails, server-side [23]) do not run;
- **work in all 17 languages** the assistant answers in;

- **not depend on negation detection**, since NegEx [12] and its successors target clinical notes in one language, while negation is sentence-final in Japanese and Korean and split in French;
- **share one vocabulary between static copy and model output.**

2. Architecture Overview

Table 1 shows one user turn on the text path. The voice path is the same up to the provider call; after that, each speakable chunk goes through steps 10 and 11 before speech.

Table 1. Turn pipeline

Phase	Operation	Purpose
1	Trim input; reject empty	—
2	Input guard : classify the current message against the lexicon's refuse groups in safety order	Answer emergencies, injuries, self-harm and rhythm questions locally; nothing is sent
3	Consent check for the active hosted provider	No health data goes to an unconsented provider
4	History assembly: drop <code>localOnly</code> turns, truncate to the provider budget, summarise dropped turns	Guarded turns never leave the device later
5	Build or reuse the fact registry; reset per-turn rate-limit counters	One address space for all facts
6	Compose tool list: 21 read tools + allow-listed actions, de-duplicated and sorted	Byte-stable schema
7	Compose system prompt: stable prefix (rules, tool overlay, capability index) + variable sections (time, user-asserted values, dashboard cache unless suppressed, live-workout marker, pending corrections, summary)	Grounding, conflict resolution, correction loop
8	Stream the provider round; tool calls go router → registry → envelope, at most 8 per turn	Retrieval with uniform absence semantics
9	Output guard, streaming : publish only complete sentences, each scrubbed	Prohibited sentences are never displayed
10	Number verification at each round end (chat) / per spoken chunk (voice)	Replace contradicted spans; record corrections
11	Output guard, speech : scrub each utterance before TTS	Nothing prohibited is spoken
12	Output guard, final pass on the whole message	Covers text assembled outside the stream
—	Build time : copy linter over Swift and string catalogues; lexicon coverage gate	Static copy follows the same vocabulary

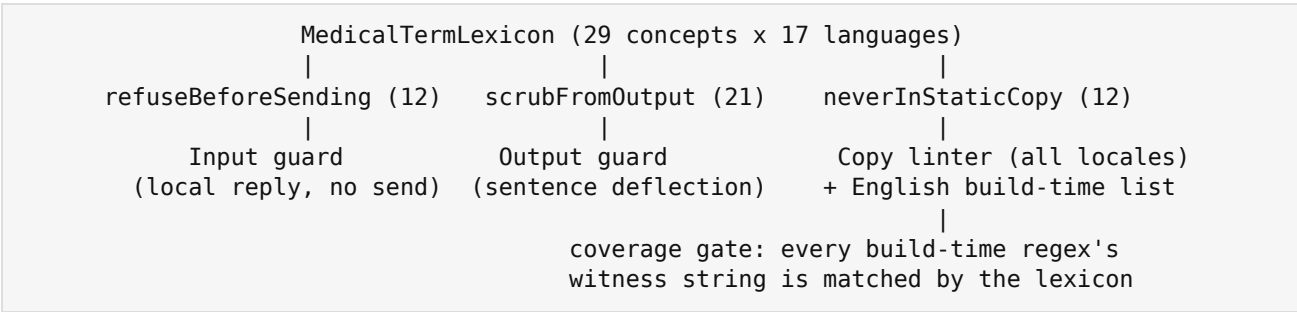


Figure 1. The shared vocabulary and its three enforcement points.

Why this shape. Each control sits where it has a deterministic answer: retrieval is uniform so a correct value is always available to quote; verification checks the one class of claim with a single ground truth; and the perimeter is one vocabulary applied wherever text crosses a boundary (into a provider, onto the screen, into the speaker, into the shipped bundle). No step depends on the model following an instruction, so the guarantees are the same for all six providers.

3. Methods in Detail

3.1 The typed fact catalogue

Entry kinds. A fact is a value of a four-case sum type `FactEntry`:

Kind	Key	Resolver signature	Notes
fixed	literal dotted key, e.g. <code>user.profile.max_hr</code>	<code>() -> FactValue, .sync or .awaitable</code>	Carries <code>valueType</code> ("Int", "Double", "Duration", "Bool", "Record"...)
parameterized	pattern with one <code>\$param</code> , e.g. <code>session.by_date(\$date)</code> , plus an example argument	<code>(param, tail) -> FactValue</code>	Matches on leading tokens; trailing tokens pass as <code>tail</code>
composite	literal or parameterised	<code>(param?, registry) -> FactValue</code>	Declares <code>dependencies</code> (atomic keys only)
action	dotted verb key, e.g. <code>routes.library.rename</code>	<code>([String:String]) -> FactValue, .sync or .awaitable</code>	Declares named <code>ActionParams</code> with a required flag; description starts with <code>[ACTION]</code> by convention

Each entry also carries an `availability` closure (3.1.4), defaulting to "always available" so a declaration is one expression. A namespace resolver exposes only `namespace: String` and `entries: [FactEntry]`; the registry derives the model-facing schema from `entries`, and the default resolve walks the same array to dispatch a key.

At e028039, 26 namespace resolvers are registered (16 for health data, 9 for app/device state and actions, and a composites namespace registered last; five share the head `app` and two `workout`). They declare **238 entries**: 149

fixed synchronous, 8 fixed awaitable, 56 parameterized, 6 composite and 19 action.

Why it works. Because the resolver closure is a field of the published entry, there is no second table to drift: a fact that can be resolved is a fact that is published, and the reverse. Adding a metric is one reviewed declaration that every provider sees on the next schema build.

3.1.1 Key grammar

```
key      := token ( "." token ) *
token    := name | name "(" argument ")"
```

1. Trim whitespace; reject empty input.
2. Scan characters, tracking parenthesis depth. A `.` at depth 0 ends a token. A `)` that would make the depth negative, a non-zero final depth, or an empty token (`a . b`, leading dot) is an error.
3. A token ending in `)` and containing `(` splits into name and argument; an empty name is an error.

Dots inside arguments survive (`session.by_date(2026.04.21)` is valid).

Pattern matching. The pattern is parsed with the same grammar. Token by token, names must be equal; a pattern argument beginning with `$` captures the key's argument (a key with no argument fails); otherwise both arguments must be equal or both absent. The pattern may not be longer than the key, remaining key tokens become `tail`, and at least one capture must occur.

Dispatch. The registry tries every namespace whose `namespace` equals the key's head token, in registration order, then all other namespaces (which is how composites resolve under keys such as `user.profile.snapshot`). No match returns `missing(notRecorded, "no such key")`.

Tool names. Dots become underscores and a `($param)` section becomes a required string property:

`session.by_date($date)` becomes `session_by_date` with `date`.

Why it works. One parse serves naming, matching and dispatch, so there is no translation layer between what the model asks for and what the resolver answers. A per-entry tool and a raw-key `lookup_fact` call reach the same resolver, and `tail` lets one parameterised entry serve sub-keys without a declaration per field.

3.1.2 Value union and envelope

`FactValue` has nine cases: `integer`, `double`, `string`, `date`, `durationSec`, `boolean`, `missing(reason, detail?)`, `list`, `record`. Every result is serialised into one sorted-key envelope:

```
{ "value": <typed JSON or null>,
  "missingReason": <reason, present only when absent>,
  "detail": <optional hint>,
  "asOf": <ISO-8601 with local UTC offset>,
  "confidence": "high" }
```

A non-finite double becomes `null` for that field only. Dates use the device's local offset, so the calendar day matches the `YYYY-MM-DD` arguments the tools accept. A nested `missing` in a list or record is rendered as an inline

envelope, reporting absence per field. On success `missingReason` is omitted, so its presence is the absence signal. `asOf` is the resolution time and `confidence` is the constant `"high"`. The cloud tool loop and the on-device dispatcher for Apple's framework call the same serialiser.

Provenance inside records. Source attribution lives in record fields where it is meaningful: `data_source` (`sport_wide` versus `route_specific` for cross-workout baselines, `live_healthkit_pending_acceptance` for sleep and vitals read before the night is accepted, with tool descriptions telling the model how to word each, e.g. "compared to your typical run" versus "on this loop"); `comparison_safe` (false when fewer than 2 prior samples back a baseline); and composite attribution (3.1.5).

3.1.3 Absence taxonomy

Every resolver that cannot return a value picks one of nine reasons; there is no free-text-only absence.

Reason	Meaning to the model
<code>notRecorded</code>	No data for this query (most common)
<code>notYetComputed</code>	Data exists; derived metric not computed yet. Retry later
<code>outOfRange</code>	Parameter outside the fact's valid range
<code>sensorDropout</code>	Signal too noisy or sparse to trust
<code>invalidParameter</code>	Malformed arguments; <code>detail</code> names the parameter and accepted range
<code>internalError</code>	Bug or budget overrun; report unavailable and stop
<code>tooMuchData</code>	Result over the size cap; narrow the query
<code>rateLimited</code>	Same call missed repeatedly this turn, or the turn's tool budget is spent
<code>partialData</code>	Composite partial result; composites carry it as <code>status: "partialData"</code> in their record (3.1.5)

Why it works. Each reason maps to a different correct next action (retry later, fix the argument, narrow the query, stop, or tell the user nothing was recorded). A free-text error leaves the model to guess which applies; a closed enumeration makes it explicit and identical across providers. Per-field absence keeps a partly available record useful.

3.1.4 No-I/O availability gating

Each entry's `availability` closure returns `{hasData, validRange?, lastUpdated?}`. It must be synchronous and metadata-only (no HealthKit, file or network access); in practice it reads in-memory archive indexes and settings flags. The schema builder runs it once per build:

1. Drop every entry with `hasData == false` from the per-entry schema.
2. For a parameterised entry with a `validRange`, append `"Example: <example>. Data available as early as <Month YYYY>; do not request earlier dates."` to the parameter description. Only the month of the lower bound is given, with no rolling end date.

The workout-backed availability, for example, is `unavailable` until the archive holds one workout, then valid from the earliest to the latest archived workout date. The filtered per-entry schema feeds the action-tool list (3.2)

and the **catalogue hash**: SHA-256 over its sorted-key JSON, recomputed on each send to detect drift in cache-relevant bytes. It changes only when an entry, a description or an availability result changes.

Why it works. Schema construction sits on the send path, so it must be fast and must not wait on I/O; metadata-only availability keeps it in memory and deterministic. Month granularity means a description changes at most once a month, which keeps the schema bytes stable for provider-side prompt caching [15] while still telling the model where data begins.

3.1.5 Composites with declared dependencies

A composite lists the atomic keys it reads; composites may not depend on composites, and a test checks that every dependency names a real entry. On the asynchronous path:

1. Substitute the composite's captured parameter (if any) for `$name` in each dependency.
2. Await each distinct dependency through the async resolver (HealthKit or network children suspend here).
3. Merge results into a registry-held prefetch map, run the synchronous body (whose `registry.resolve(key)` reads the map first), then restore the previous map. Nothing suspends in between, so no other resolve sees the map.
4. The body returns `{present: [{key, value}], missing: [{key, reason, detail?}]}`, adding `status: "partialData"` when any child is missing.

A synchronous walk that reaches an un-prefetched awaitable child returns `missing(internalError, "resolves asynchronously ...")`; it neither blocks nor traps.

Why it works. Declared dependencies separate the slow part (awaiting children) from the logic (a synchronous body over known values), which keeps the body simple and testable. The scoped map cannot leak to concurrent resolves, and the `{present, missing}` record tells the model exactly which part of a summary is backed by data.

3.1.6 Registry rails

Every tool result passes a final gate, in order:

1. **Repeat-miss limit.** Signature `toolName + "::" + argsJSON`. Each *missing* result increments a per-turn counter; above 2, the result becomes `missing(rateLimited, "Same tool+args returned missing 2 times this turn. Stop retrying...")`. Successful results do not count; counters reset each user turn.
2. **Size cap.** A non-missing result whose envelope exceeds 80,000 UTF-8 bytes becomes `missing(tooMuchData, "... Ask a narrower query...")`.
3. **Time budget.** On the per-entry path, resolution over 2.0 s is discarded as `missing(internalError, "resolver timed out...")`; over 0.5 s logs a warning, and over 4.0 s on an action logs a watchdog warning. On the compact read-tool path the budget only logs, because legitimate HealthKit and network waits can exceed 2 s.
4. **Numeric range checks.** Each numeric parameter is declared once with a closed range, an optional excluded lower bound and an integer flag. Only a finite in-range number (whole where flagged) is accepted; anything else, including "inf", "nan" and "1e20", returns `invalidParameter` stating the requirement, e.g. "ordinal must be a whole number from 0 to 100000, got '-1'". Comma-separated blobs must have exactly the expected field count, empty fields counted (60, ,5 has three).

5. **Required action arguments.** A missing one returns `invalidParameter` naming it, before the action runs; unknown keys are dropped.

Why it works. Every rail answers in the same envelope, so the model is steered rather than stopped: it stops retrying, narrows a query or fixes an argument, and the turn still ends in a useful reply. Counting only misses per exact signature targets the loop that wastes a turn without penalising legitimate repeated reads, and stating the accepted range in words lets the model correct the call on the next round.

3.2 Compact tool layer

The model sees 39 tools, not 238:

- **21 read tools** with fixed names and string parameters, most taking a selector (`which`, `field`, `metric` or `aspect`) that routes to catalogue keys: `get_today`, `get_session`, `get_recovery`, `get_hrv`, `get_sleep`, `get_vitals`, `get_workout`, `list_workouts`, `get_walks`, `get_baseline`, `get_user`, `get_app_state`, `get_training_load`, `get_score_meta`, `get_breadcrumbs`, `get_routes`, `get_tags`, `get_workout_live`, `get_assistant_state`, `get_healthkit`, and `lookup_fact` for any raw catalogue key.
- **18 action tools** under their catalogue names (e.g. `assistant_email_compose`, `routes_library_rename`, `web_search`, `directions_routeTo`), taken from the availability-filtered per-entry schema through an explicit allow-list. Of the 19 declared actions, `location.roads_ahead` is not allow-listed.

Read tools are appended first, then actions; duplicates by name are dropped (first wins, with a warning, because one provider rejects duplicates); the list is sorted by name. Action schemas list `required` sorted; every property is a string and the resolver converts types.

Capability index. The read-tool list also generates a system-prompt section: each tool's name and the first sentence of its description, capped at 170 characters, with an instruction to call the matching tool before answering any question about the user's own data.

Retrieval and caps. A BM25 ranker [13] ($k_1 = 1.5$, $b = 0.75$, target 40) runs over the compact schema; at 39 tools it returns the list unchanged. The on-device provider, whose context holds only a few tools, uses the BM25 order to choose which to keep. A per-provider cap (110 for one provider) truncates the ranked list.

Per-turn budget. When `callsSoFar + callsThisRound > 8`, every call in that round gets `missing(rateLimited, "tool budget exceeded for this turn")`; the model has one more round to answer in text, and further tool calls are dropped. The on-device dispatcher enforces the same constant.

Why it works. A few polymorphic read tools are easier for every provider to choose among than hundreds of narrow ones, and they fit the on-device context. Catalogue names keep every side effect identifiable in audit logs. Sorting and de-duplication make the schema bytes a pure function of the catalogue, so the stable prefix caches well, and generating the index from the same list means the prompt always describes the tools that exist.

3.3 Runtime number verification

Family	Metrics	Source of truth	Where applied
Live workout	HR, power, DFA $\alpha 1$, HR drift	Live workout snapshot	Voice, each chunk before speech
App state	TSB, ATL, CTL, ACWR, RMSSD, recovery score, last night's sleep duration and efficiency	Training-load registry (same accessor as dashboard and tools); latest reliable overnight session; latest overnight with sleep	Chat, every tool-loop round end; voice, each chunk before speech

Table 2. Claim shapes and tolerances

Metric	Claim shape (case-insensitive, simplified)	Tolerance	Replacement
HR	"your [current/live] HR/heart rate [is/s]..." or "HR right now/currently/now", filler, $\backslash d\{2,3\}$ bpm	5 bpm	N bpm
Power	same shape, "power/wattage", $\backslash d\{2,4\}$ W/watts	15 W	N W
DFA $\alpha 1$	same shape, " $\alpha 1$ /alpha 1", optional "DFA", decimal only	0.10	%.2f
HR drift	same shape, "[HR] drift", number + %	1.5 points	%.1f%
TSB	TSB + optional "negative" + signed number	2.0	%+.1f
ATL / CTL	label + number	3.0	%.1f
ACWR	label + number	0.15	%.2f
RMSSD	RMSSD [of] N [ms]	8.0 ms	%.0f ms
Recovery	recovery N	4.0 on 0–100; a claim ≤ 10 within 1.0 of the 0–10 score is accepted	on the claim's scale
Sleep duration	"slept [for] [about] Nh [M m]", "sleep [duration] was ...", "... of sleep"	5 min if minutes or decimal hours given; 30 min for whole hours	"6h 55m" or "6 hours and 55 minutes", matching the claim
Sleep efficiency	"[sleep] efficiency [of/was/...] N%", "N% [sleep] efficient/efficiency", in a sentence mentioning sleep	1.0 point	N%

App-state patterns reject a number followed by a time or count unit (days, h, weeks, nights, minutes, sessions, workouts), by %, or by more digits, so "CTL 42-day window" is not a claim.

Algorithm (app-state family).

- Normalise spelled numbers.** Match $\backslash b(negative\backslash s+)?(ones-or-teen | tens([-\backslash s]+digit)?)(\backslash s+point\backslash s+digit)?\backslash b$, alternations longest first, and replace with digits ("negative thirty-one point

- seven" → -31.7), recording (originalRange, normalizedRange) rewrites. Range 0–99, at most one decimal digit, English words.
2. Run the metric regexes on the normalised text. Each names a **value** group and a **claim** group (number plus unit, the span to replace); the metric label is never in the replaced span.
 3. Find each match's sentence (the enumerator does not split "17.2"). Discard it if the sentence has a **time marker** (ago, yesterday, last week/month/.../weekday, previous(ly), earlier, before, on) or a **target/comparison marker** (average, mean, baseline, typical, usual, target, goal, keep, stay, peak, max, min, trend, would, could, if, should, threshold, under, below, above, over, between, range, higher, lower, than, and similar).
 4. Negate a "negative N" claim. Keep the match only if $|claimed - actual|$ exceeds the tolerance.
 5. Recovery: drop the match if $|claimed - score10| \leq 1.0$; report a claim ≤ 10 on the 0–10 scale.
 6. Sleep: drop matches in sentences with general-norm markers ("need", "recommended", "adults", "per night", ranges such as "7-9") or other-night markers (month and weekday names, "nights", "weeks", dates); drop overlaps.
 7. **Remap** ranges to the original text; an offset inside a rewritten span snaps to the rewrite's original start or end, so "negative thirty-one point seven" is replaced whole.
 8. Replace claimed spans back-to-front. Every other word, including unrelated spelled numbers ("two sessions"), is unchanged.

The live-workout family runs steps 2–4 without normalisation and requires the "your X is" / "X right now" shape, because a bare "150 bpm" is usually a target or a different metric.

Application. In chat, verification runs at the end of every tool-loop round, including rounds ending in a tool call, before the pre-tool text is rewound; the corrected text replaces the saved turn. In voice, each chunk passes output scrub → live-workout verification → app-state verification → preamble → speech normalisation, so the corrected number is the one spoken.

Why it works. A claim about the value *now* has exactly one ground truth in app state, so it can be checked deterministically; history, averages, targets and norms do not, and the markers keep them out of scope so the verifier corrects only what it can know. Tolerances follow dashboard rounding and each metric's noise, so rounded or approximate statements stand. Replacing only the number-and-unit span keeps the model's sentence and reasoning intact. Normalising spelled numbers and remapping offsets let one set of regexes serve chat and voice, and the dual-scale recovery rule and precision-matched sleep tolerance accept the ways people naturally state those values.

Correction loop. Each discrepancy enters a process-wide, lock-guarded buffer, de-duplicated by (metric, claimed, actual) (voice sees the same claim in the spoken chunk and the saved turn) and capped at the 4 most recent. The next system prompt reads and clears it into a variable section outside the cached prefix:

```
# Last turn correction – DO NOT FABRICATE
Your previous response contained numbers that contradicted live data. The saved reply was
corrected, but the user may already have read or heard the wrong number – if it matters,
correct it briefly. For any of these metrics next turn, CALL the appropriate tool (...)
and
quote what comes back – never estimate or interpolate.
• You said TSB=-29.0 – actual was -15.5.
```

Why it works. One fabrication yields exactly one reminder on the next turn, naming the metric, both values and the tool to call, so the next answer is grounded without re-generating the current one. Consume-once and the cap keep the prompt short, and placing the section outside the prefix keeps the cache intact.

3.4 Structural resolution of user-asserted values

Before the variable prompt sections are built, a detector walks the last 6 turns newest first, keeping user turns that are not `localOnly`. For each message:

1. Normalise spelled numbers (3.3 step 1).
2. Remove every sentence ending in `?` or `?`; questions assert nothing.
3. For each known metric name (training load, HRV, heart rate, recovery, sleep, workout and vitals terms; longest first so "max hr 190" is not read as "hr 190"), find the *last* `\bmetric\b\s*[connector]\s*number`. The connector is optional except for `sleep` and `recovery`, which need one of `is/of/=/:/was/being`. Clock times (5 am, 11 pm, 5:30, o'clock) are excluded. Matches are blanked after use; the newest assertion per metric wins. Transcription variants (`bsb`, `tsp`, `tbs` → `tsb`, ...) fold into the canonical key only if the canonical spelling gave no value.
4. Set `dashboardContradicted` on a contradiction phrase ("the dashboard says", "real numbers are", "you're wrong", "wrong number", ...) or a sentence opening "no/actually, it's/it is" followed by a number or a word such as not/different/wrong/higher/lower/more/less.
5. Set `explicitOverrideRequested` on phrases such as "stop calling tools", "use these numbers", "I don't care what the dashboard...", "stop saying".

Policy: if any signal is present, inject `# User-stated values (authoritative – fresher than any cache)` with one `- key: value` line per assertion (training-load metrics first), adding a stale-cache sentence on contradiction or override. If `dashboardContradicted` or `explicitOverrideRequested`, **omit the dashboard training-load block entirely**.

Why it works. A model given two conflicting numbers and an instruction to prefer one still holds both, and may average them or pick the wrong one. Removing the stale block leaves nothing to reconcile. Reading statements only, newest first, with a required connector for ambiguous words, keeps questions and clock times from being taken as assertions.

3.5 The multilingual regulated-claim lexicon

Concept structure. A concept has an `id`, Latin-script alternatives and "unbounded" alternatives, compiled as

```
(?: \b(?:latin_1|latin_2|...)\b | (?:unbounded_1|unbounded_2|...) )
```

case-insensitively against the original text (never a case- or diacritic-folded copy), so match ranges align with the string being rewritten. Accented forms are spelled out (`m[ée]decin`). CJK text has no inter-word boundary (a `\b` around 心房細動 never matches in running Japanese), so CJK and Arabic alternatives match as substrings.

Languages. English plus ar, da, de, es, fi, fr, is, it, ja, ko, nb, nl, pt-BR, ru, sv and zh-Hans: 17. A test fails if any concept lacks non-Latin alternatives.

Concepts (29). Rhythm: atrial fibrillation, arrhythmia, irregular heartbeat. Conditions: named cardiac condition, acute cardiac event, neurovascular event. Claims: diagnosis, cure, treatment claim, prescription, regulatory clearance, pathology, clinical physiology labels, symptom of disease, speculative diagnosis. Risk framing: injury risk, overtraining, risk-zone framing. Certainty: categorical autonomic state, physiological certainty, unsupported metric verdict. Referral: medical referral. Input-side: chest pain, breathlessness, syncope, severe symptom, danger judgement, self-harm, injury. A separate two-concept offensive-language list uses the same structure.

Benign-homonym exclusions, written per alternative because the qualifier's position differs by language:

Benign sense	Exclusion
Respiratory sinus arrhythmia (normal physiology)	EN (?<sinus\s)arrhythmias?; FR arhythmies?(?!s+sinusales?); ES/PT aritmias?(?!s+sinusal(?es)?); NL/DE/DA/NB/FI (?<sinus)...; RU (?<синусовая\s)(?!синусовой\s)аритми[яи]; JA (?<洞性)不整脈; ZH (?<窦性)心律失常; KO (?<동성)(?!동성)부정맥; AR negative lookahead for / التنفسي الجيبي . German Sinusarrhythmie passes because the compound has no inner \b
Software diagnostics ("Diagnostic log")	`diagnos...(?!s+(?:log
Swimming "stroke rate", "stroke of luck"	no bare stroke ; only `(?;having
"Suicide sprints" (a drill)	`su[ïi]cide(?!s+(?:sprints?
Finnish <i>parantaa</i> ("improve"), Italian <i>prenditi cura</i> ("take care"), Icelandic <i>greining</i> ("analysis"), Germanic <i>Rezept/recept</i> ("recipe")	only disease-object or prescription-only shapes match
"This is your recovery score"	(?-i:[Tt]his\s+IS\s+your) is case-sensitive inside a case-insensitive pattern, so only the emphatic claim matches

Groups.

- `refuseBeforeSending` (12) = rhythm (3) + emergency (chest pain, breathlessness, syncope, severe symptom, self-harm, acute cardiac event, neurovascular event) + general concern (danger judgement) + injury.
- `scrubFromOutput` (21): all rhythm, condition, claim, risk, certainty and referral concepts.
- `neverInStaticCopy` (12): rhythm (2), condition (3), regulatory clearance, cure, prescription, treatment, speculative diagnosis, pathology, clinical physiology labels.

Eight concepts are **input-only**. Chest pain, breathlessness, syncope, severe symptom, self-harm and injury, because the model must be able to say these words when explaining why the app cannot help. Danger judgement, because its bare adjectives (`dangerous`, `gefährlich`) appear in the app's own negating educational copy ("not because the number is dangerous in itself"); assertion-shaped danger framing is covered on output by risk-zone framing. Acute cardiac event, a strict subset of the already-scrubbed named cardiac condition, exists so the input guard can refuse "am I having a heart attack?" without refusing "is my resting HR bradycardia?", an ordinary endurance-athlete question. Every concept is in one of the two roles, or both (rhythm and neurovascular). `neverInStaticCopy` excludes concepts the app's copy uses on purpose: the localized redirect "Consult a doctor if consistently low"

matches `medicalReferral`, and the whole lexicon produced 45 such matches over the catalogue, while the 12 selected concepts matched nothing in any locale, so the group can act as a hard gate.

Why it works. One vocabulary consumed at three points means a term forbidden in shipped copy is also handled at input and in model output, in every language, and one edit reaches all three. Splitting by script gives each language the boundary rule that fits it; matching on the original text keeps replacement ranges exact; per-language exclusions at the qualifier's natural position let normal physiology, software terms and sports vocabulary through. Per-concept group membership lets each enforcement point act without rewriting the app's own safety language.

3.6 Input guard

Order (first match wins): 1. self-harm → crisis reply; 2. emergency group → symptom reply; 3. injury → injury reply; 4. rhythm → rhythm reply; 5. danger judgement → symptom reply; 6. none → proceed.

Replies are fixed catalogue strings, localized into all 17 languages:

- Crisis: an apology, "please reach out to someone now", the local emergency number, findahelpline.com, and 988 for the US. No mention of a doctor.
- Symptom: "If you're exercising, stop now. Talk to your doctor about that. I can't assess your health — Emuqu is a fitness coaching app, not a medical device. If you're feeling unwell, please contact a clinician (or your local emergency number for severe symptoms)."
- Injury: stop, call the local emergency number if serious, the app cannot assess injuries, otherwise see a clinician.
- Rhythm: the app does not detect AFib or arrhythmia and is not a medical device; talk to your doctor; a clinically validated ECG feature exists for that purpose.

On a refusal: (1) append the user turn and the canned reply, both flagged `localOnly`, and persist them; (2) return without calling any provider, including the on-device one, and before the consent check. The flag lives on the stored turn, so history assembly withholds such turns before budgeting (they are never counted as "dropped"), summarisation filters them again and the correction detector skips them; a guarded message cannot reach a provider later as history.

Feature flag. The guard defaults on; the self-harm branch runs even when it is off: `if flagOn || classify(text) == selfHarm.`

Prompt parity. System-prompt rule B asks the model to reply exactly with the symptom, self-harm and injury texts, and unit tests check that the symptom and injury replies appear verbatim (after whitespace collapse) in both the cloud and on-device prompts.

Voice echo trap. The spoken rhythm refusal itself contains "AFib" and "arrhythmia", so a transcript is dropped as an echo when it would trigger the refusal that is already the last assistant turn and at least 40% of its tokens overlap that reply.

Why it works. An emergency, injury or self-harm message needs a correct answer at once and identically every time, so a fixed translated string answers it, not a model. The order puts the most urgent resource first: self-harm leads because its reply must carry a crisis line rather than a doctor referral; an acute symptom outranks rhythm

because its reply names an emergency number, so "skipped beats and chest tightness" does not get the smartwatch-ECG pointer; a risk-judgement request ranks below rhythm because "should I be worried about my AFib" is best served by the ECG pointer. Running before consent and flagging the stored turn keep the message on the device now and later, and prompt parity gives the user the same words whichever path answers.

3.7 Output guard

Scope. Sentences are deflected when they name a condition for the user, speculate about one, assess rhythm, claim cure, treatment, prescription or regulatory clearance, frame risk as an injury prediction or "danger zone", assert categorical autonomic certainty, or rank a metric as truth. General physiology passes: HRV, parasympathetic activity, respiratory sinus arrhythmia, training load, sleep. Prompt rules A and D tell the model not to give dosing or personal prescriptions; on output, prescription vocabulary is matched.

Negation-neutral deflections. Each deflection is true whether the replaced sentence affirmed or denied the term, e.g. "Emuqu measures beat-to-beat timing; it doesn't assess heart rhythm. A clinician or a clinically validated ECG is the right place for that question." Deflections are catalogue strings in the in-app language; a test checks that none trips any rule. There are 13 medical deflections (9 concept-specific, 4 shared: rhythm, load, out-of-scope, observation fallback) plus one offensive-language line.

```
scrub(text):
    active = medicalRules + offensiveRule (+ englishOnlyRule if it matches and reply is
    English)
    if text empty or firstMatch(text, active) is none: return text unchanged
    segments = split(text) so that concat(segments) == text exactly
    for each segment:
        core = segment without leading/trailing whitespace
        rule = firstMatch(core, active)
        if rule: emit leadingWS + rule.deflection + trailingWS; record (rule.reason, core)
        else:    emit segment byte-identical
    if nothing was replaced:          # match straddles a terminator
        m = rule.regex.firstMatch(original text)
        rebuilt = text with every segment overlapping m collapsed into ONE deflection
        if m found and firstMatch(rebuilt, active) is none: return tidy(rebuilt)
        else: return rule.deflection  # whole message
    return tidy(rebuilt)              # " "->" ", " .->".", " ,->","

firstMatch(t, rules):
    r = first rule whose regex matches t
    if r: return r
    plain = t with every '*', '_', '`' removed
    return plain != t ? first rule matching plain : none
```

Segmentation. A segment runs through a terminator from `{. ! ? \n . ! ? , ? ...}`, any further terminators (`?! , ...`), then whitespace. A `.` after a digit is a decimal point when a digit follows; at the end of the text it is undecided and does not split. A sentence is rewritten at most once and rewritten text is never re-scanned, so the pass is linear with no iteration cap. The markdown retry catches emphasis inside a term (You may have **atrial** fibrillation.); callers needing a range re-run the regex on the original text and fail closed.

Where it runs. (1) **Streaming buffer** (chat and the voice cursor): deltas publish at most every 33 ms; each publish splits at the last terminator, scrubs only the complete part and holds the tail back, so a prohibited phrase never

renders even partially. The voice "speaking cursor" advances only over published text. At round end and on every exit path, including errors and Stop, the tail is force-published, scrubbed. (2) **Speech**: every utterance (streamed chunks, completed turns, error lines, wake-word acknowledgements) passes one function, scrub → number verification → preamble, before the synthesiser. (3) **Final pass** over the whole message after the stream ends.

Why it works. A denial is a finding too ("This isn't AFib" rules a condition out), and negation is spread across sentences in ways one pattern cannot follow in 17 languages (sentence-final in Japanese and Korean, split in French, a negative verb in Finnish). Deflections true in both readings remove the need to decide which reading applies, so the guard is correct in every language without a negation model. Whole-sentence replacement keeps replies grammatical; holding back tails keeps a prohibited phrase off screen even for a frame; one speech function leaves no path to the speaker that skips the guard; and the straddle rule makes the boundary case fail closed.

3.8 Build-time copy linter

The linter scans the app source tree, the Watch app and both string catalogues:

1. **Swift literal scanner.** One pass handles `//` comments, *nested* `/* */` comments, `"""` multi-line literals and raw literals (`#"..."#`, `##"..."##`, where backslash is not an escape), blanking interpolations `\(expr)`. Measured on this code base, a quote-pairing regex missed 93% of the characters inside multi-line literals, including all long-form help articles; the scanner reads them.
2. **Prohibition-context exemption.** The system prompt quotes forbidden framings to forbid them ("Never tell the user that a number 'predicts injury'..."). A match is excused only when (a) a guard such as `never|don't|do not + tell|say|use|describe|call|label|refer|frame|claim|imply` matches within a 3-line window of the **comment-stripped** source and the literal's first 40 characters (at least 3) lie in that same sentence, or (b) the match's own sentence inside the literal contains a guard. A comment cannot excuse a string; catalogue values are never excused this way.
3. **Catalogues.** An allow-listed catalogue *key* clears its translations too. Every catalogue is checked against the English build-time list (62 patterns) and, in *every locale*, the multilingual `neverInStaticCopy` concepts, which the linter parses from the Swift lexicon source (bracket-matched `latin` and `unbounded` arrays, unescaped one level, compiled with the runtime's `\b` rules).

A missing scan path, unreadable file, unparseable catalogue, missing `neverInStaticCopy` group or zero usable concepts, or an uncompileable pattern each exits with code 2. PDF reports and the post-workout Coach Report email body are built from templates, contain no LLM text, and are governed by this linter.

Why it works. Static copy ships in the bundle and never passes the runtime guards, so it is checked where it is written. Scanning literals as the compiler reads them covers long-form text, not just one-line strings. Scoping the exemption to the prohibiting sentence on comment-stripped source lets the prompt name what it forbids without letting a nearby comment excuse other copy. Reading the multilingual set from the runtime source means every locale follows the model's vocabulary with no second copy to keep in step.

3.9 Coverage gate

The CI gate proves the runtime lexicon matches every build-time term:

1. Extract every literal from every `latin:/unbounded:` array in all lexicon files (a glob, so a file split hides nothing); bracket matching skips string literals, so a `]` inside a character class does not close the array. Compile each, word-bounding Latin alternatives.
2. **Fail (exit 2)** if any alternative matches the empty string or fewer than 100 were extracted; either means the parser is broken. At e028039 there are 985.
3. For each build-time pattern, **synthesise a witness**: remove `(?i)` and lookarounds; `\s+` → space; remove `\s*` and `\b`; reduce character classes to one member (`\s` → space, `\x` → `x`); drop leftover quantifiers; repeatedly reduce `(a|b)?` → `""`, `(a|b)` → `a`, `(a)?` → `""`, `(a)` → `a`; remove `?`.
4. **Fail (exit 2)** if the witness does not match its own pattern.
5. **Fail (exit 1)** if no lexicon alternative matches the witness, listing each gap.

Why it works. Two hand-kept vocabularies drift. A witness turns each build-time pattern into a concrete example the runtime lexicon must recognise, so a term added to one list only fails CI. The self-check and the vacuity guards ensure the gate cannot pass because its own parser broke.

3.10 Consent and provider abstraction (context)

Each hosted provider needs its own consent before its first send; the consent key includes a schema version (currently 8) bumped when the disclosure changes, and removing an API key revokes consent. The on-device provider is exempt. On failure, the fallback chain holds only other enabled, available, consented hosted providers, then the on-device provider. An on-device safety refusal is shown as refused and not retried elsewhere. Each outbound request is recorded before provider transforms in an in-memory first-in-first-out audit of 10 entries.

4. What Is New

Each statement is the specific rule that stands after a prior-art search made on 2026-10-08 (agent-framework code and documentation, patent databases, arXiv, vendor documentation, public code hosting). "Builds on" names the nearest public work and what the rule adds. "Earliest dated record" gives the first commit in the author's repository lineage (Section 8), and the date of the current form where later. Repositories: flow-recovery-old (Jan 2026), flow-recovery (Mar–Aug 2026), emuqu-dev (Aug–Sep 2026), emuqu (public, from 2026-09-08).

N1. A fact catalogue keyed by literal dotted keys or patterns with exactly one `$param`, parsed by one depth-aware grammar that keeps dots inside parentheses. The same parse yields the tool name (dots → underscores; (`$param`) → required string property), a leading-token match that passes remaining tokens as a sub-key, and dispatch (head-token namespaces first, in registration order, then all others), so a per-entry tool and a raw-key `lookup_fact` reach the same resolver.

Builds on: the OpenAI Agents SDK, which derives a tool's schema and dispatch from one declaration [17] (not claimed); adds a dotted key-pattern grammar shared by naming, matching and dispatch.

Earliest dated record: flow-recovery c63d56c, 2026-04-23 ("major AI voice upgrades"); depth-aware split emuqu-dev 680bbec, 2026-08-25.

N2. A per-entry availability check that is synchronous and metadata-only (no HealthKit, files or network), returning `{hasData, validRange, lastUpdated}`. The schema builder drops entries without data and appends "Data available as early as ; do not request earlier dates" to each parameterised entry, giving only the month of the lower

bound; the filtered schema is hashed (SHA-256, sorted-key JSON) to detect drift in cache-relevant bytes.

Builds on: the OpenAI Agents SDK `is_enabled` option (PR #808, 2025-06-03), which hides a tool per run [17] (not claimed); adds the metadata-only constraint and month-granular range text.

Earliest dated record: flow-recovery c63d56c, 2026-04-23 (same rule).

N3. A tool-result envelope whose absence channel is a closed nine-value enumeration with a defined meaning per value; `missingReason` is omitted on success, so its presence is the signal; nested `missing` values are serialised as inline envelopes, reporting absence per field; one serialiser feeds five hosted providers and the on-device dispatcher.

Builds on: the OpenAI Agents SDK, which returns tool exceptions (`failure_error_function`) and, since 2026-02-13, timeouts (`timeout_behavior="error_as_result"`) to the model as tool output [17] (not claimed); adds a closed absence taxonomy with per-field nesting across six providers.

Earliest dated record: flow-recovery c63d56c, 2026-04-23 (nine-case taxonomy identical); on-device path flow-recovery e36fee4, 2026-05-06.

N4. A registry gate: (a) a per-turn counter on the exact (`tool name`, `arguments JSON`) signature, incremented only by `missing` results, turns the third identical miss into a `rateLimited` absence telling the model to stop retrying, without halting the run; (b) a non-missing envelope over 80,000 UTF-8 bytes becomes `tooMuchData`; (c) numeric arguments declared once with a closed range, optional excluded lower bound and integer flag reject out-of-range, non-finite, non-integral and wrong-field-count input (empty fields counted) as `invalidParameter` stating the accepted range in words. The wall-clock budget between (a) and (b) is not claimed.

Builds on: per-tool timeouts returned as results (OpenAI Agents SDK [17]) and the OpenHands stuck detector, which halts after four repeated action–observation pairs or three repeated erroring actions [26]; adds miss-only, per-signature steering through an absence, with size and range rails in the same envelope.

Earliest dated record: flow-recovery c63d56c, 2026-04-23 (miss counter, 80 KB cap); range-checked arguments emuqu c1d875e, 2026-10-05.

N5. Composite facts that declare atomic dependencies, awaited into a registry-held prefetch map that a synchronous body reads and that is restored before anything can suspend, returning a `{present, missing}` record with per-child absence reasons and `status: "partialData"` when any child is missing.

Builds on: dependency-declared computation in general; no specific public prior art was found. Adds scoped prefetch for a synchronous body and per-child attribution returned to the model.

Earliest dated record: flow-recovery c63d56c, 2026-04-23 (present/missing records); scoped prefetch emuqu 554540f, 2026-10-03.

N6. A two-tier tool surface: 21 fixed polymorphic read tools plus 18 allow-listed actions taken from the availability-filtered catalogue under their original names, de-duplicated by name (first wins) and sorted for byte stability, with a system-prompt capability index generated from the same read-tool list (first sentence, capped at 170 characters).

Builds on: tool calling [3][4][14] and BM25 ranking [13]; no specific prior art was found for the two-tier surface. Adds the fixed read tier over a catalogue, audit-preserving action names and a generated index.

Earliest dated record: flow-recovery d5d6180, 2026-05-06 (compact router); de-duplication and sorting emuqu-dev 680bbee, 2026-08-25; capability index present at emuqu-dev d4e5cf8, 2026-08-16.

N7. A free-text numeric verifier that (a) checks only present-value claims, discarding a match whose sentence has a time or target/comparison marker or whose number is followed by a time or count unit; (b) normalises English spelled numbers (0–99, one decimal, optional "negative") and maps offsets back, snapping to rewritten spans so "negative thirty-one point seven" is replaced whole; (c) replaces only the number-and-unit span; (d) checks recovery on both scales (a claim ≤ 10 within 1.0 of the 0–10 score is accepted); (e) sets sleep-duration tolerance by claim precision (5 min, or 30 min for whole hours); (f) runs on each voice chunk before speech and at every chat tool-loop round end.

Builds on: Proof-Carrying Numbers [18] (claim-bound tokens verified under a tolerance policy, fail-closed); US Patent 12,670,149 [19] (detecting and correcting inaccurate numeric comparisons); PHIA [21] (code-computed answers from wearable data); US Patent 12,353,469 B1 [20] (per a search extract, citation-grounded answers). Tolerance checks against structured values and pre-return correction are not claimed. Adds spelled-number parsing in free speech, present-value scoping by sentence markers and span-only replacement before speech.

Earliest dated record: flow-recovery 1b5ad2a ("hallucination guard"), 2026-05-02 (live voice chunks, tolerances, span replacement); app-state claims 10fc926, 2026-05-17; spelled numbers e8ca54a, 2026-06-02; time/target markers emuqu 554540f, 2026-10-03.

N8. A correction loop: discrepancies de-duplicated by (metric, claimed, actual), capped at four, consumed exactly once into a variable section of the *next* turn's prompt outside the cached prefix, telling the model the user may have seen the wrong number and to call the named tool; the reply is not re-generated.

Builds on: US Patent 12,670,149, which re-prompts to fix a detected numeric error [19] (not claimed); adds consume-once, de-duplicated next-turn feedback outside the cache.

Earliest dated record: flow-recovery 68f32e5 ("hallucination feedback"), 2026-05-08 (cap 4, consume once); de-duplication emuqu 5c4a23d, 2026-10-04.

N9. Structural resolution of user-versus-cache conflicts: statements (not questions) in the last six non-local user turns that assert metric values or contradict the cache cause the cached training-load block to be removed and a user-stated block injected, instead of instructing the model to prefer the user.

Builds on: the knowledge-conflict literature on model behaviour; adds resolution by prompt construction.

Earliest dated record: flow-recovery e8ca54a, 2026-06-02 (same mechanism).

N10. One regulated-claim lexicon of 29 concepts in seventeen languages, split into word-bounded Latin alternatives and substring CJK/Arabic alternatives, with per-language, position-aware lookarounds for benign homonyms (e.g. sinus arrhythmia in each script), consumed through three named groups (input refusal, output scrubbing, never-in-static-copy) with per-concept membership.

Builds on: Llama Guard 3, a multilingual safety classifier with Specialized Advice and Self-Harm categories [22] (not claimed); adds a shared regex lexicon with homonym exclusions driving three enforcement points.

Earliest dated record: emuqu-dev 8c3999d, 2026-08-26; English-only precursors flow-recovery 69ee32d (2026-04-26) and 0a3972d (2026-05-03).

N11. An input guard that (a) classifies in the order self-harm, acute emergency, injury, rhythm, danger judgement; (b) answers with a fixed catalogue-localized reply that unit tests show the cloud and on-device prompts quote verbatim; (c) persists both turns flagged `localOnly` on the stored turn, honoured by history assembly (before budgeting), summarisation and the assertion detector; (d) sends nothing to any provider and runs before consent; (e) answers self-harm even with its feature flag off.

Builds on: Myra's crisis protocol, which screens every message before the AI with a phrase list and classifier and

replies with a fixed message and region-localized crisis lines [24], and a public Hugging Face Space with a keyword crisis reply [25]; a pre-model crisis screen with a fixed reply is not claimed. Adds ordered multi-class refusal, local-only persistence across history and tested prompt parity.

Earliest dated record: flow-recovery 69ee32d, 2026-04-26 (English, nothing sent); `localOnly` flow-recovery 9645baa, 2026-05-01; ordered classes emuqu-dev 8c3999d, 2026-08-26; self-harm class emuqu 3585687, 2026-09-30; injury class emuqu c1d875e, 2026-10-05.

N12. An on-device, negation-neutral output guard replacing whole sentences with deflections true in both readings; retrying on markdown-stripped text; collapsing a sentence-straddling match into one deflection, or deflecting the whole message if that still matches; running on complete sentences before display (holding back tails with a decimal-aware split), on every spoken utterance, and in a final pass.

Builds on: NeMo Guardrails streaming output rails, server-side [23], and NegEx [12]; checking streamed output before delivery is not claimed. Adds negation-neutral sentence deflection on the device before display and speech.

Earliest dated record: flow-recovery 0a3972d, 2026-05-03 (English sentence deflection); negation-neutral multilingual form emuqu-dev 8c3999d, 2026-08-26; decimal-aware split emuqu 5c4a23d, 2026-10-04.

N13. A build-time copy linter with a real Swift literal scanner (nested comments, multi-line and raw literals, blanked interpolations), a prohibition exemption evaluated on comment-stripped source within the prohibiting sentence only, key-level allow-listing that covers translations, multilingual patterns parsed from the runtime lexicon source and applied to every locale, and fail-closed input handling.

Builds on: regular-expression keyword filtering; no specific prior art was found. Adds the scanner, sentence-scoped exemption and lexicon-sourced multilingual check.

Earliest dated record: flow-recovery 0a3972d, 2026-05-03 (Swift string scanner, comment stripping, string catalogues); prohibition context emuqu-dev 59ce5da, 2026-08-26; multilingual patterns emuqu-dev 8c3999d, 2026-08-26.

N14. A coverage gate that synthesises a witness for each build-time regex, verifies it against its own pattern, requires a runtime-lexicon alternative to match it, and refuses to pass when any alternative matches the empty string or fewer than one hundred were extracted.

Builds on: regex string generation (exrex [27]; Hypothesis `from_regex` [28]) (not claimed); adds its use to prove one vocabulary covers another, with self-check and vacuity guards.

Earliest dated record: emuqu-dev 8c3999d, 2026-08-26 (same rule).

Not claimed (prior art). Model tool or function calling [3][4][14]; deriving tool schema, dispatch and enablement from one declaration [17]; returning tool errors and timeouts to the model as results [17]; loop detection on repeated agent actions [26]; retrieval-augmented generation [2]; provider-side prompt caching [15]; BM25 ranking [13]; regular-expression keyword filtering in general; classifier- or rail-based LLM guardrails, including multilingual classifiers and streaming output rails [10][11][22][23]; pre-model crisis screening with a fixed reply [24][25]; verification or correction of model numbers against structured values with tolerances, and re-prompting on an error [18][19]; post-hoc claim verification with model calls [5][6]; clinical negation detection [12]; generating strings from a regular expression [27][28]; multi-provider fallback; per-vendor consent prompts; instruction hierarchies [9][16].

5. Verification

These methods are software controls, not physiological measurements. The science register (Tools/science_register/register.json, scoring version v3.1.oct2026) classifies the app's physiological heuristics; none of its entries covers a method here, and none is described as validated. The independent validation report for the companion papers' physiological methods is Tools/validation/RESULTS.md. Verifier tolerances are set from dashboard rounding and metric noise; detection rates on real model output are not claimed.

Behaviour is fixed by unit tests in EmuquTests (test-function counts at e028039) and by CI gates:

Suite	Tests	Establishes
EmuquTests/CoachVoiceGuardTests.swift	44	200 identical prohibited sentences all rewritten; 100 clean sentences byte-identical among 100 prohibited; "you may have atrial fibrillation" intercepted in all 16 non-English locales; sinus arrhythmia passes in every locale; straddles fail closed; decimals do not split; nothing prohibited published mid-stream
EmuquTests/CoachVoiceGuardNegationTests.swift	4	Denial and assertion of a term get the same neutral line; every translated deflection exists and passes
EmuquTests/MedicalQueryGuardTests.swift	29	Rhythm and symptom questions refused in every locale; crisis reply for self-harm in every locale; "suicide sprints" proceed; symptom outranks rhythm; rule B quotes the reply verbatim; every refuseBeforeSending concept refused, with lexicon-derived probes
EmuquTests/MedicalQueryGuardTests+Injury.swift	6	Injury class and reply
EmuquTests/LocalOnlyTurnTests.swift	7	localOnly persists; refused turns withheld from history, not counted as dropped, also under truncation
EmuquTests/HallucinationFeedbackTests.swift	15	Buffer caps at four, clears on consume; wrong current HR, duration, efficiency corrected without the label; targets, norms, other nights, averages ignored; spelled durations checked; right or rounded values kept
EmuquTests/FactValueTests.swift	41	No missingReason on success; non-finite nulls only its field; nested missing envelopes; sorted keys; valid JSON
EmuquTests/FactCatalogValidationTests.swift	8	Unique, provider-valid tool names; valid placeholders; real composite dependencies; fast availability; deterministic schema; every key round-trips
EmuquTests/FactNumericArgumentTests.swift	9	Hostile numeric values and field counts rejected with the range named, without trapping

`Tools/copy_linter/lint.py` and `scripts/check_perimeter_sync.sh` run in CI, and `scripts/verify_gates_fail.sh` plants a violation for each (a localized "This can diagnose your arrhythmia." and a malformed catalogue for the linter; a term added to one list only for the coverage gate) and requires failure. At e028039 the coverage gate probes 62 patterns against 985 alternatives, clean.

6. Design Tradeoffs

The perimeter is lexical by choice (deterministic, on-device, auditable), so a paraphrase that avoids the vocabulary is not caught and obfuscation beyond removing emphasis markers is not normalised.

Negation-neutral deflection buys correctness in all 17 languages at the cost of also deflecting correct denials ("there's no sign of overtraining"), and a sentence with a matched term is replaced whole.

The guards take precedence over prompt rules that permit general descriptions of conditions, so the coach does not deliver general information about named conditions.

The input guard reads the current message only, keeping its decision immediate; a question split across turns is not joined.

Number verification covers present-value claims for the 11 metrics in Table 2, in English, so historical values, averages and non-English replies are not corrected, and live-workout claims are checked on the voice path.

In chat the correction lands at the end of the round, after its text may have been displayed, and the next-turn reminder addresses that.

The 21 compact read tools are static for byte stability, so availability gating applies to the per-entry schema and the action tools.

A round that would exceed the 8-call budget has all its calls refused, which keeps the rule simple and leaves the model one clean round to answer.

The coverage gate is one-directional: build-time terms must be covered at runtime, while the runtime lexicon may be broader.

Model-drafted email bodies go to the system mail composer for the user to review and send, and auto-extracted memories are written by a background model call; both sit outside the output guard.

7. Comparison to Existing Approaches

Product and library facts are limited to their public documentation, code or filings; (search extract) marks items seen only through a search-engine extract on 2026-10-08.

Approach	Documented behaviour	Overlap	Difference	Ref.
Function calling / tool use (OpenAI, Anthropic)	Model may call declared tools; output not checked	Tool layer (3.2)	No absence taxonomy, number check or perimeter	[14] [15]
OpenAI Agents SDK	<code>function_tool</code> builds schema and dispatch from one declaration; <code>is_enabled</code> (2025-06-03) hides tools per run; <code>failure_error_function</code> and <code>timeout_behavior="error_as_result"</code> (2026-02-13) return failures as tool output	N1–N4 in general form	No key-pattern grammar, metadata-only range text, closed absence enum or per-signature miss limit	[17]
OpenHands stuck detector	Halts after 4 identical action–observation pairs or 3 identical erroring actions	N4	Emuqu counts only misses per signature per turn and returns an absence; it does not halt	[26]
RAG	Retrieved passages in context	Grounding	No output check	[2]
RARR / FActScore	Post-hoc claim check with model calls and retrieval	Verification	Model calls after generation; evaluated primarily in English	[5] [6]
Proof-Carrying Numbers (Sep 2025)	Claim-bound numbers verified against a source under a tolerance policy; fail-closed	N7	Uses claim-bound tokens; Emuqu parses free text and spelled numbers, scopes to present values, replaces only the span	[18]
US Patent 12,670,149	Detects and corrects inaccurate numeric comparisons; re-prompts to fix (search extract)	N7, N8	Present-value filtering, voice-chunk span replacement and consume-once feedback not described in the material seen	[19]
PHIA (2024)	Agent answering wearable-data questions by generating and running code	Personal-data grounding	No post-generation number check or perimeter described	[21]

Approach	Documented behaviour	Overlap	Difference	Ref.
NeMo Guardrails	Programmable input/output rails; streaming output rails check chunks before the client	N12 timing	Server-side; no negation-neutral deflection or decimal-aware buffering documented	[11] [23]
Llama Guard / Llama Guard 3	Classifier over prompt and response; Llama Guard 3: 8 languages, S6 Specialized Advice, S11 Self-Harm	N10, N11	Model call per check; general taxonomy; no build-time link	[10] [22]
Myra crisis protocol	Every message screened before the AI (phrase list + classifier); fixed message plus region-localized crisis lines	N11	No ordered multi-class refusal, local-only history persistence or prompt-parity test documented	[24]
Keyword crisis gate (Hugging Face Space)	Fixed crisis message on a keyword match	N11	English keyword list	[25]
NegEx-style rules	Negation detection in clinical notes	— (not claimed)	Emuqu does not use negation detection	[12]
exrex; Hypothesis from_regex	Generate strings matching a regex	N14 witness step	No cross-vocabulary coverage proof or vacuity guards	[27] [28]
WHOOOP Coach	GPT-4 "fine-tuned with anonymized member data and proprietary WHOOOP algorithms"; zero retention (search extract)	Health-data coach	Nothing public on number verification, guardrails or languages	[29]
Oura Advisor	LLM with Oura's algorithms; inputs include scores, contributors, tags, profile and past interactions (search extract); "not a medical device"; medical questions can be routed to Counsel Health's clinicians	Health-data coach; medical boundary	Nothing public on number verification or output filtering	[30] [31] [32]
Emuqu (this paper)	Typed catalogue with closed absence taxonomy; present-value verifier with span correction and next-turn feedback; one 17-language lexicon at input, output and build time	—	Regex on device, no model call; checks before display and speech	—

8. Provenance and Dates

Project origin. The following records establish when the project began. They are origin evidence, not dates for any method in this paper.

- Repository `chrissharp80/AI-Fitness-Coach`: 19 commits, 2025-02-16 to 2025-02-17, all by Chris Sharp. It read and displayed Apple HealthKit heart rate and HRV and streamed heart rate from a Polar H10 over Bluetooth. It contains none of the methods in this paper.
- The author's ChatGPT history contains the earliest recovered HRV-specific design exchange: a user message on 2025-02-22 at 15:44:18 UTC proposing an app that, with a Polar strap worn to bed, would "learn right before they're gonna wake up and then administer the test ... similar to what WHOOP tries to do, but ... actually get it right". The excerpt was recovered through conversation-history retrieval on 2026-10-08; the full transcript and a stable conversation ID are not exposed. The same day's discussion lists "HRV trends" in a fitness-app feature set. Messages on 2025-03-04 and 2025-03-05 cover the build/deploy/test workflow and a VO_2 -max validation plan. None of these concern the assistant.

Repository lineage. All repositories are private except `emuqu`. The author retains all of them with full history.

Repository	First commit	Notes
<code>flow-recovery-old</code> ("Flow HRV")	2026-01-12	250 commits to 2026-01-26
<code>flow-recovery-dev</code>	2026-03-01	Squashed import
<code>flow-recovery</code> ("Flow Recovery")	2026-03-15	196 commits to 2026-08-12
<code>emuqu-dev</code>	2026-08-16	Squashed import; to 2026-09-04
<code>emuqu</code> (public, github.com/chrissharp80/emuqu , PolyForm Strict 1.0.0)	2026-09-08	Public

Earliest commit per statement.

N	First form (commit, date)	Current form (commit, date)
N1	<code>flow-recovery</code> c63d56c, 2026-04-23	<code>emuqu-dev</code> 680bbec, 2026-08-25 (depth-aware split)
N2	<code>flow-recovery</code> c63d56c, 2026-04-23	same
N3	<code>flow-recovery</code> c63d56c, 2026-04-23	<code>flow-recovery</code> e36fee4, 2026-05-06 (on-device path)
N4	<code>flow-recovery</code> c63d56c, 2026-04-23	<code>emuqu</code> c1d875e, 2026-10-05 (range-checked arguments)
N5	<code>flow-recovery</code> c63d56c, 2026-04-23	<code>emuqu</code> 554540f, 2026-10-03 (scoped prefetch)
N6	<code>flow-recovery</code> d5d6180, 2026-05-06	<code>emuqu-dev</code> 680bbec, 2026-08-25 (de-dup and sort)
N7	<code>flow-recovery</code> 1b5ad2a, 2026-05-02	<code>emuqu</code> 554540f, 2026-10-03 (time/target markers)
N8	<code>flow-recovery</code> 68f32e5, 2026-05-08	<code>emuqu</code> 5c4a23d, 2026-10-04 (de-duplication)
N9	<code>flow-recovery</code> e8ca54a, 2026-06-02	same
N10	<code>flow-recovery</code> 69ee32d, 2026-04-26 (English precursor)	<code>emuqu-dev</code> 8c3999d, 2026-08-26
N11	<code>flow-recovery</code> 69ee32d, 2026-04-26	<code>emuqu</code> c1d875e, 2026-10-05 (injury class)

N	First form (commit, date)	Current form (commit, date)
N12	flow-recovery 0a3972d, 2026-05-03	emuqu 5c4a23d, 2026-10-04 (decimal-aware split)
N13	flow-recovery 0a3972d, 2026-05-03	emuqu-dev 8c3999d, 2026-08-26 (multilingual patterns)
N14	emuqu-dev 8c3999d, 2026-08-26	same

Dated non-git records (the author's own Google Drive). Documents dated 2026-05-06 and uploaded on 2026-05-10 (README 2.md, VOICE_AND_TOOL_USE 2.md, flow-recovery-audit.md, audit-results-20260506) describe the fact catalogue as the tool schema (about 200 tools, 19 namespaces). They also describe availability-closure gating with the valid range in parameter descriptions, and a uniform {value, missingReason, asOf, confidence} envelope across Apple and cloud providers. The registry rails are there too: the third identical miss becomes rateLimited, an 80 KB cap gives tooMuchData, and resolvers have a 2 s budget. So are composite {present, missing} records, [ACTION] tools, the input guard and output regex guard, and a copy-linter term list with the (?<!sinus\s) lookbehind on "arrhythmia". These agree with the git dates above. Earlier Drive records under the names "Flow HRV" (report, 2026-01-12) and "Flow Recovery" (session reports 2026-02-16, 2026-02-18, 2026-03-04) concern measurement, not the assistant.

Public disclosures.

- TestFlight beta from 2026-01-27; 45 testers by August 2026. A public TestFlight join link appears in the project README by May 2026.
- Substack, "A Powerfully Lazy Man's Way to Better Health", 2026-05-19. It described dual internal-plus-streamed RR capture merged in the morning, DFA $\alpha 1$ "organized windows", and a 30–70% sleep search band picking the highest-RMSSD window. It described no method in this paper.
- Substack, "How I Build Production Software by Directing AI", 2026-08-10. It described the author's AI-directed development process, the beta (45 testers, over 6,500 sessions) and a readiness score, but no method in this paper.
- Public repository github.com/chrissharp80/emuqu, first commit 2026-09-08.
- These white papers, v1.0, v1.1 and v1.2, 2026-10-08.
- Before any public release, the work was shared privately with two colleagues, in January and March 2026.

Authorship. Chris Sharp chrissharp80@gmail.com is the sole author and sole committer. He directs AI coding assistants to write the code, and some commits are attributed to "Claude"; he supplies the requirements, design direction, review, testing and acceptance.

This paper: v1.2, published 2026-10-08, describing source revision e028039 (2026-10-07).

References

1. Ji, Z., Lee, N., Frieske, R., et al. (2023). Survey of Hallucination in Natural Language Generation. *ACM Computing Surveys* 55(12).
2. Lewis, P., Perez, E., Piktus, A., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems (NeurIPS)* 33.

3. Schick, T., Dwivedi-Yu, J., Dessì, R., et al. (2023). Toolformer: Language Models Can Teach Themselves to Use Tools. *NeurIPS* 36.
4. Yao, S., Zhao, J., Yu, D., et al. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. *International Conference on Learning Representations (ICLR)*.
5. Gao, L., Dai, Z., Pasupat, P., et al. (2023). RARR: Researching and Revising What Language Models Say, Using Language Models. *Proceedings of ACL 2023*.
6. Min, S., Krishna, K., Lyu, X., et al. (2023). FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation. *Proceedings of EMNLP 2023*.
7. U.S. Food and Drug Administration (2019). *General Wellness: Policy for Low Risk Devices*. Guidance for Industry and FDA Staff.
8. Apple Inc. *App Store Review Guidelines*, Guideline 1.4.1 (Physical Harm — medical apps). Public documentation.
9. Greshake, K., Abdelnabi, S., Mishra, S., et al. (2023). Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*.
10. Inan, H., Upasani, K., Chi, J., et al. (2023). Llama Guard: LLM-based Input-Output Safeguard for Human-AI Conversations. arXiv:2312.06674.
11. Rebedea, T., Dinu, R., Sreedhar, M., Parisien, C., Cohen, J. (2023). NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications with Programmable Rails. *Proceedings of EMNLP 2023: System Demonstrations*.
12. Chapman, W. W., Bridewell, W., Hanbury, P., Cooper, G. F., Buchanan, B. G. (2001). A Simple Algorithm for Identifying Negated Findings and Diseases in Discharge Summaries. *Journal of Biomedical Informatics* 34(5).
13. Robertson, S., Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval* 3(4).
14. OpenAI (2023). Function calling. OpenAI API documentation.
15. Anthropic (2024). Tool use and prompt caching. Anthropic API documentation.
16. Wallace, E., Xiao, K., Leike, R., et al. (2024). The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions. arXiv:2404.13208.
17. OpenAI. OpenAI Agents SDK for Python: `function_tool`, `is_enabled` (pull request #808, merged 2025-06-03), `failure_error_function`, and `timeout_behavior="error_as_result"` (added 2026-02-13), in `src/agents/tool.py`. <https://github.com/openai/openai-agents-python> (accessed 2026-10-08).
18. Solatorio (2025). Proof-Carrying Numbers (PCN): A Protocol for Trustworthy Numeric Answers from LLMs via Claim Verification. arXiv:2509.06902. <https://arxiv.org/abs/2509.06902> (accessed 2026-10-08).
19. U.S. Patent No. 12,670,149. Detection and correction of inaccurate numeric comparisons in LLM responses. United States Patent and Trademark Office. <https://image-ppubs.uspto.gov/dirsearch-public/print/downloadPdf/12670149> (accessed 2026-10-08; assignee and filing date not recorded in this search).
20. U.S. Patent No. 12,353,469 B1 (Amazon). <https://patents.google.com/patent/US12353469B1> (accessed 2026-10-08; seen via search extract only).
21. Merrill, M. A., et al. (2024). Transforming Wearable Data into Health Insights using Large Language Model Agents (PHIA). arXiv:2406.06464. <https://arxiv.org/abs/2406.06464> (accessed 2026-10-08).
22. Meta (2024). Llama Guard 3 model card. <https://huggingface.co/meta-llama/Llama-Guard-3-8B> (accessed 2026-10-08).

23. NVIDIA. NeMo Guardrails documentation: streaming and output rails. <https://docs.nvidia.com/nemo/guardrails/> (accessed 2026-10-08).
24. Myra. Myra — Crisis Response Protocol. <https://sites.google.com/view/myra-privacy-policy/home/crisis-resource> (accessed 2026-10-08).
25. Deva1211/chatbot, Hugging Face Space, app.py. <https://huggingface.co/spaces/Deva1211/chatbot/blob/main/app.py> (accessed 2026-10-08).
26. OpenHands. Agent stuck detector. <https://docs.openhands.dev/sdk/guides/agent-stuck-detector> (accessed 2026-10-08).
27. exrex: irregular methods on regular expressions (Python package). <https://pypi.org/project/exrex/> (accessed 2026-10-08).
28. Hypothesis documentation, `hypothesis.strategies.from_regex`. <https://hypothesis.readthedocs.io/> (accessed 2026-10-08).
29. OpenAI. WHOOP customer story. <https://openai.com/customer-stories/whoop> (accessed 2026-10-08; seen via search extract only).
30. Oura. Oura Advisor (support article). <https://support.ouraring.com/hc/en-us/articles/39512345699219> (accessed 2026-10-08).
31. Oura (2025-03-31). Oura Advisor, an AI-powered Personal Health Companion, Now Rolling Out to All Oura Members. Business Wire. <https://www.businesswire.com/news/home/20250331565896/en/> (accessed 2026-10-08).
32. Oura. Counsel integration in the Oura app (blog). <https://ouraring.com/blog/counsel-integration-oura-app/> (accessed 2026-10-08).

Appendix A: Parameter Table

Name	Value	Unit	Role
Registered namespace resolvers	26	count	Registry address space
Declared catalogue entries	238 (149 fixed sync, 8 fixed async, 56 parameterised, 6 composite, 19 action)	count	Catalogue size at e028039
Absence reasons	9	count	Closed taxonomy
<code>rateLimitMissRetries</code>	2	misses	Third identical miss per turn → <code>rateLimited</code>
<code>outputSizeCapBytes</code>	80,000	bytes	Per-result cap → <code>tooMuchData</code>
<code>defaultResolveBudgetSec</code>	2.0	s	Per-entry path: overrun discards value
<code>resolveWarnBudgetSec</code>	0.5	s	Slow-resolver warning
<code>watchdogWarnSec</code>	4.0	s	Action main-thread warning
Valid-range text granularity	month of lower bound	—	Parameter description
Compact read tools	21	count	Model-facing schema

Name	Value	Unit	Role
Allow-listed action tools	18	count	Model-facing schema
Capability-index purpose cap	170	characters	Per tool line
BM25 k1, b	1.5, 0.75	—	Tool ranking
BM25 target K	40	tools	Retriever filters only above this
Grok schema cap	110	tools	Per-provider cap
<code>maxToolCallsPerTurn</code>	8	calls	Over-budget round → all calls <code>rateLimited</code>
Stream publish interval	33	ms	Throttle; sentence-complete publishing
HR tolerance	5	bpm	Verifier
Power tolerance	15	W	Verifier
DFA α 1 tolerance	0.10	—	Verifier
HR drift tolerance	1.5	percentage points	Verifier
TSB tolerance	2.0	load units	Verifier
ATL / CTL tolerance	3.0	load units	Verifier
ACWR tolerance	0.15	ratio	Verifier
RMSSD tolerance	8.0	ms	Verifier
Recovery tolerance	4.0 (0–100) / 1.0 (0–10)	points	Verifier, scale disambiguation
Sleep duration tolerance	5 (precise) / 30 (whole hours)	min	Verifier
Sleep efficiency tolerance	1.0	percentage points	Verifier
Spelled-number range	0–99, one decimal digit	—	Normaliser
Pending corrections cap	4	entries	Next-turn reminder
Correction-detector window	last 6 turns (user, non-local)	turns	Assertion detection
Numeric arg: <code>starting_atl</code> , <code>starting_ctl</code> , <code>daily_trimp</code>	0–10,000	TRIMP	Range check
Numeric arg: <code>gap_trimp</code>	(0, 10,000]	TRIMP	Range check
Numeric arg: <code>horizon_days</code>	1–365, integer	days	Range check
Numeric arg: <code>lat</code> / <code>lon</code>	–90–90 / –180–180	degrees	Range check
Numeric arg: <code>radius_m</code>	5–5,000	m	Range check
Numeric arg: <code>ordinal</code>	0–100,000, integer	index	Range check
Numeric arg: <code>seconds</code>	1–86,400, integer	s	Range check
Numeric arg: <code>max_results</code>	1–10, integer	results	Range check

Name	Value	Unit	Role
Lexicon concepts	29 (+2 offensive)	count	Shared vocabulary
Languages	17	count	en + 16 shipped locales
<code>refuseBeforeSending / scrubFromOutput / neverInStaticCopy</code>	12 / 21 / 12	concepts	Group sizes
Sentence terminators	<code>. ! ? \n 。 ! ? , ? ...</code>	—	Segmentation
Emphasis markers stripped on retry	<code>`* _ ``</code>	—	Output guard
Offensive-language recogniser confidence	0.6	probability	English-only entries
Offensive-language min words	3	words	English-only entries
Voice echo overlap	0.40	token fraction	Refusal echo trap
Linters prohibition window	3 lines (match line and 2 above)	lines	Exemption
Linters snippet fragment	first 40 chars, ≥ 3	characters	Exemption
Build-time English patterns	62	count	<code>user_facing_prohibited</code>
Coverage-gate minimum alternatives	100 (985 at e028039)	count	Vacuity guard
Consent schema version	8	—	Per-provider consent
Request audit size	10	entries	In-memory FIFO

Appendix B: Pseudocode

B.1 Registry tool resolution

```

resolveTool(name, argsJSON):
  entry = findEntry(toolName == name) or return missing(invalidParameter, "unknown tool")
  if entry is action:
    args = parse(argsJSON) keeping only declared params, numbers/bools -> strings
    for p in entry.params where p.required and p not in args:
      return gate(name, argsJSON, missing(invalidParameter, "missing required arg
p"))
    v = await entry.execute(args)
    return gate(name, argsJSON, v)
  key = entry.key, or entry.pattern with "$param" replaced by args[param]
    (missing/bad arg -> missing(invalidParameter))
  t0 = now; v = await resolveAsync(key); dt = now - t0
  if dt > 2.0: v = missing(internalError, "timed out")
  return gate(name, argsJSON, v)

gate(name, argsJSON, v):
  if v is missing:
    sig = name + ":@" + argsJSON; count[sig] += 1
    return count[sig] > 2 ? missing(rateLimited, ...) : v
  if utf8len(envelope(v)) > 80000: return missing(tooMuchData, ...)
  return v

```

B.2 Composite (async path)

```

compositeAsync(entry, key):
  param = entry.key has "(" ? capture(entry.key, key) : nil
  deps = entry.dependencies with "$name" -> param
  values = { d: await resolveAsync(d) for d in unique(deps) }
  saved = prefetch; prefetch = prefetch u values
  r = entry.body(param, registry)      # body's registry.resolve(k) reads prefetch
first
  prefetch = saved
  return r                             # {present:[{key,value}], missing:
[{{key,reason,detail}}, status?}

```

B.3 App-state number verification

```

verifyAppState(text):
  (norm, rewrites) = normaliseSpelledNumbers(text)
  found = []
  for (pattern, actual, tol, fmt, negateOnSpelled) in metricChecks:
    for m in pattern.matches(norm):
      s = sentenceAround(m, norm)
      if s matches timeMarkers or targetMarkers: continue
      c = number(m.value); if negateOnSpelled and m.claim contains "negative": c = -
|c|
      if |c - actual| > tol: found += (metric, fmt(c), fmt(actual), m.claimRange)
  apply recovery-scale rule and sleep sentence filters
  mapped = [remap(d.range, norm -> text, rewrites) for d in found]
  corrected = replace mapped ranges back-to-front in text with d.actual
  recordCorrections(mapped)          # dedupe (metric, claimed, actual); keep last 4
  return corrected

nextPromptVariableSections():
  block = takeAndClear(pendingCorrections)
  if block nonempty: append "# Last turn correction – DO NOT FABRICATE" + one line per
entry

```

B.4 User-assertion resolution

```

signals = {}
for msg in reversed(lastSixTurns.filter(user and not localOnly)):
  t = statementsOnly(normaliseSpelledNumbers(msg))
  for metric in knownMetricsLongestFirst:
    v = lastMatch(metric, connector required if metric in {sleep, recovery}, no clock
times)
    if v and metric not in signals.values: signals.values[metric] = v; blank matches
fold aliases into canonical if canonical absent
  signals.contradicted |= containsPhrase(t) or startsWithCorrectionReply(t)
  signals.override     |= containsOverridePhrase(t)
if signals nonempty: prompt += userStatedBlock(signals)
if not (signals.contradicted or signals.override): prompt += dashboardLoadBlock

```

B.5 Input guard

```

onSend(text):
  t = trim(text); if empty: reject
  trig = first of [selfHarm, emergency, injury, rhythm, dangerJudgement] whose regexes
match t
  if trig and (flagOn or trig == selfHarm):
    store(userTurn(t, localOnly=true), assistantTurn(replyFor(trig, appLanguage),
localOnly=true))
    return          # no provider, no consent prompt
  continue to consent check and provider send
history(turns) = turns.filter(not localOnly and not emptyAssistant)

```

B.6 Streaming output guard

```

onDelta(chunk):
  buf += chunk
  if now - lastPublish < 33ms: return
  (complete, tail) = splitAtLastTerminator(buf)  # decimal-aware
  if complete empty: return
  buf = tail
  visible += scrub(complete)
  if roundHasNoToolCall and published text has terminator: speakableCursor = len(visible)
onRoundEnd / onError / onStop:
  visible += scrub(buf); buf = ""
speak(utterance):
  synthesise(preamble + verify(scrub(utterance)))

```

B.7 Coverage gate

```

alts = [compile(latin ? "\b(?:\"+a+)\b" : a) for a in literals(latin/unbounded arrays of
lexicon*)]
fail(2) if any alt matches "" or len(alts) < 100
for p in buildTimePatterns:
  w = witness(p)                                # see 3.9 step 3
  if w == "": continue
  fail(2) if not p matches w
  gaps += p if no alt matches w
fail(1) if gaps else pass

```

Appendix C: Source Map

Method	Files (repository-relative)
Entry kinds, one-line-one-fact, namespace walk, pattern matching	Emuqu/Sources/Assistant/Facts/FactCatalog.swift
Key grammar	Emuqu/Sources/Assistant/Facts/FactKey.swift
Value union, absence taxonomy, availability, envelope	Emuqu/Sources/Assistant/Facts/FactValue.swift
Registry, rails, schema, hash, composites, timeout race	Emuqu/Sources/Assistant/Facts/FactResolverRegistry.swift
Numeric argument ranges	Emuqu/Sources/Assistant/Facts/AppFactResolver+NumericArguments.swift
Namespace registration; entries	Emuqu/Sources/Assistant/Facts/AppFactResolver.swift, AppFactResolver+*.swift
Provenance flags (data_source, comparison_safe)	AppFactResolver+TrainingProjections.swift, AppFactResolver+WorkoutLiveDerived.swift, AppFactResolver+Sleep.swift

Method	Files (repository-relative)
Compact tools, allow-list, capability index	Emuqu/Sources/Assistant/Facts/CompactToolRouter.swift, CompactToolRouter+Resolvers.swift
BM25 retrieval	Emuqu/Sources/Assistant/Facts/ToolRetriever.swift
Tool loop, call budget, chat verification, fallback	Emuqu/Sources/Assistant/ViewModel/AssistantViewModel+Tools.swift
On-device dispatcher	Emuqu/Sources/Assistant/Providers/AppleToolDispatcher.swift
Number verifier, correction buffer	Emuqu/Sources/Assistant/MetricsVerifier.swift
Voice verification and scrub order	Emuqu/Sources/Assistant/VoiceConversationController+Speech.swift
User-assertion detector	Emuqu/Sources/Assistant/UserAssertedValuesParser.swift
Prompt composition (cache suppression, corrections)	Emuqu/Sources/Assistant/Providers/AIProvider+SystemPrompt.swift; rules A–D in AIProvider+SystemPromptText.swift
Lexicon and groups	Emuqu/Sources/Assistant/MedicalTermLexicon.swift, MedicalTermLexicon+Injury.swift, MedicalTermLexicon+MetricVerdict.swift
Input guard	Emuqu/Sources/Assistant/MedicalQueryGuard.swift; call site Emuqu/Sources/Assistant/ViewModel/AssistantViewModel.swift; echo trap VoiceConversationController+Audio.swift
localOnly turns	Emuqu/Sources/Assistant/Providers/AIProvider.swift (ChatTurn); Emuqu/Sources/Assistant/ViewModel/AssistantViewModel+Context.swift
Output guard, offensive list	Emuqu/Sources/Assistant/CoachVoiceGuard.swift
Streaming buffer	Emuqu/Sources/Assistant/ViewModel/AssistantViewModel+Context.swift (StreamTextBuffer)
Final pass	Emuqu/Sources/Assistant/ViewModel/AssistantViewModel+Routing.swift
Copy linter and configuration	Tools/copy_linter/lint.py, Tools/copy_linter/prohibited_terms.json
Coverage gate	scripts/check_perimeter_sync.sh
Consent	Emuqu/Sources/Assistant/ProviderConsentTracker.swift
Tests cited	EmuquTests/MedicalQueryGuardTests.swift, MedicalQueryGuardTests+Injury.swift, CoachVoiceGuardTests*.swift, FactCatalogValidationTests.swift
Validation register	Tools/science_register/register.json